



中国海洋大学
信息科学与工程学部

Lab4 ICMP 协议分析实验报告

课程名称 计算机网络

指导教师 洪峰

学生姓名 刘和雄

学号 24070001071

专业/班级 24 软工

学年学期 2025 秋季学期

提交日期 2025.12.03



目录

1	引言	2
2	ICMP 的 ping 指令分析	2
2.1	ping bilibili.com 测试结果	2
2.2	抓包分析 ICMP 数据包	2
2.3	问题解答	4
3	ICMP 的 traceroute 分析	6
3.1	traceroute 相关图片	6
3.2	问题解答	8
4	实验结果与分析	10
4.1	ICMP 协议分析总结	10
5	结论与展望	11

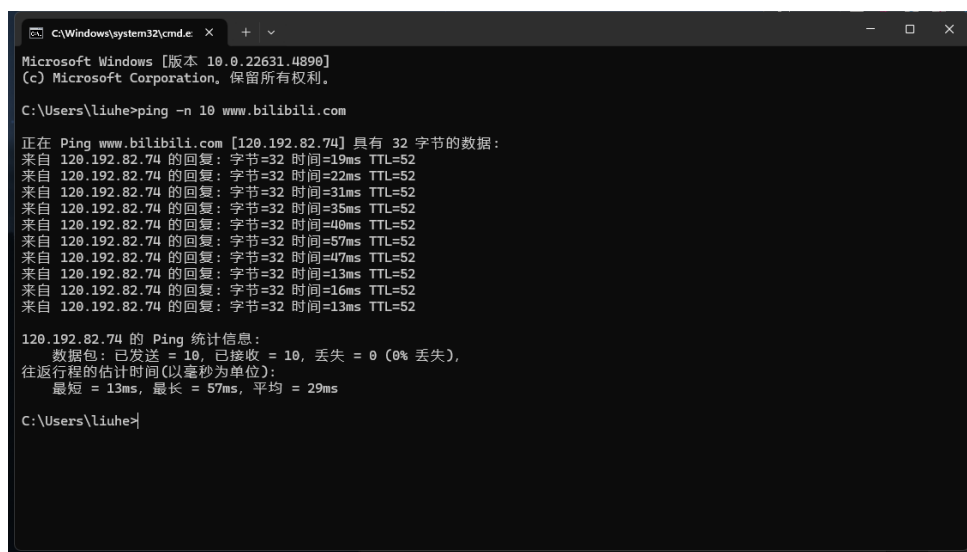
1 引言

本实验旨在分析 ICMP (Internet Control Message Protocol) 协议的工作原理，通过使用 ping 和 traceroute 工具来观察 ICMP 数据包的结构和行为。

2 ICMP 的 ping 指令分析

2.1 ping bilibili.com 测试结果

在本次实验中，我们对 bilibili.com 进行了 ping 测试，以观察 ICMP Echo Request 和 Echo Reply 数据包的交互过程。



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.22631.4890]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\liuhe>ping -n 10 www.bilibili.com

正在 Ping www.bilibili.com [120.192.82.74] 具有 32 字节的数据:
来自 120.192.82.74 的回复: 字节=32 时间=19ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=22ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=31ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=35ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=40ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=57ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=47ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=13ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=16ms TTL=52
来自 120.192.82.74 的回复: 字节=32 时间=13ms TTL=52

120.192.82.74 的 Ping 统计信息:
    数据包: 已发送 = 10, 已接收 = 10, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 13ms, 最长 = 57ms, 平均 = 29ms

C:\Users\liuhe>
```

图 1: ping bilibili.com 命令执行结果

2.2 抓包分析 ICMP 数据包

为了分析 ICMP 数据包的结构，我们使用 Wireshark 对 ping 过程进行了抓包。



2.2.1 ICMP 请求报文结构

The image shows a Wireshark packet capture of ICMP Echo (ping) requests and replies. The top pane displays a list of packets, and the bottom pane shows the detailed structure of a selected ICMP Echo (ping) request packet.

No.	Time	Source	Destination	Protocol	Length	Info
52	0.000000	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=6/1536, ttl=128 (reply in 53)
72	0.994389	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=7/1792, ttl=128 (reply in 73)
90	0.983071	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=8/2048, ttl=128 (reply in 91)
133	0.977712	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=9/2304, ttl=128 (reply in 13..)
159	0.974817	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=10/2560, ttl=128 (reply in 1..)
274	0.971204	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=11/2816, ttl=128 (reply in 2..)
327	0.949854	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=12/3072, ttl=128 (reply in 3..)
352	0.964118	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=13/3328, ttl=128 (reply in 3..)
404	0.995743	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=14/3584, ttl=128 (reply in 4..)
449	1.005363	10.150.161.101	120.192.82.74	ICMP	74	Echo (ping) request id=0x0001, seq=15/3840, ttl=128 (reply in 4..)
53	0.018991	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=6/1536, ttl=52 (request in 5..)
73	0.022619	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=7/1792, ttl=52 (request in 7..)
91	0.031638	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=8/2048, ttl=52 (request in 9..)
136	0.034916	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=9/2304, ttl=52 (request in 1..)
160	0.040531	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=10/2560, ttl=52 (request in ..)
278	0.057288	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=11/2816, ttl=52 (request in ..)
331	0.047765	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=12/3072, ttl=52 (request in ..)
353	0.013188	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=13/3328, ttl=52 (request in ..)
405	0.016633	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=14/3584, ttl=52 (request in ..)
450	0.013125	120.192.82.74	10.150.161.101	ICMP	74	Echo (ping) reply id=0x0001, seq=15/3840, ttl=52 (request in ..)

The bottom pane shows the detailed structure of the selected ICMP Echo (ping) request packet (No. 52):

- Ethernet II, Src: Intel_9b:3d:d1 (e4:60:17:9b:3d:d1), Dst: IETF-VRRP-...
- Internet Protocol Version 4, Src: 10.150.161.101, Dst: 120.192.82.74
- Internet Control Message Protocol
 - Type: 8 (Echo (ping) request)
 - Code: 0
 - Checksum: 0x4d55 [correct]
 - [Checksum Status: Good]
 - Identifier (BE): 1 (0x0001)
 - Identifier (LE): 256 (0x0100)
 - Sequence Number (BE): 6 (0x0006)
 - Sequence Number (LE): 1536 (0x0600)
 - [Response frame: 53]
 - Data (32 bytes)

Hex dump of the packet data:

```
0000 00 00 5e 00 01 01 e4 60 17 9b 3d d1 08 00 45 00
0010 00 3c 8f 78 00 00 80 01 00 00 0a 96 a1 65 78 c0
0020 52 4a 08 00 4d 55 00 01 00 06 61 62 63 64 65 66
0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76
0040 77 61 62 63 64 65 66 67 68 69
```

图 2: ICMP 请求报文 (Echo Request) 结构

2.2.2 ICMP 回应报文结构

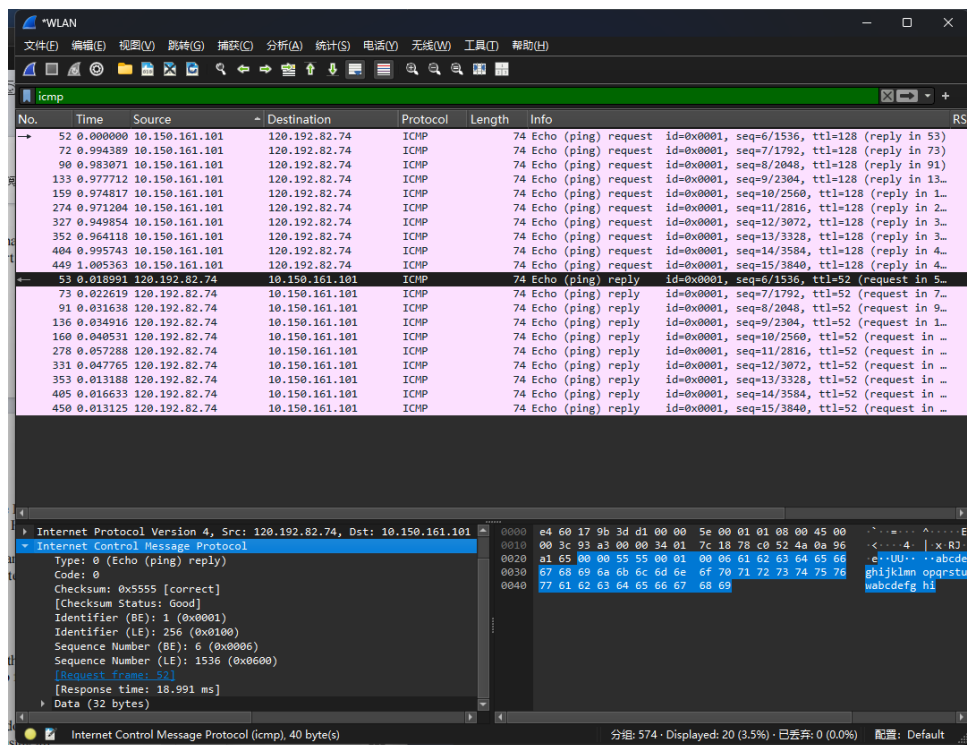


图 3: ICMP 回应报文 (Echo Reply) 结构

2.3 问题解答

2.3.1 问题 1

问题: What is the IP address of your host? What is the IP address of the destination host?

回答: 根据 Wireshark 抓包分析:

- 本机 (源) IP 地址: 10.150.161.101
- 目标 (目的) IP 地址: 120.192.82.74

2.3.2 问题 2

问题: Why is it that an ICMP packet does not have source and destination port numbers?



回答：端口号（Port Numbers）是**传输层**（Transport Layer，如 TCP 或 UDP）的概念，用于区分同一台主机上运行的不同应用程序（进程）。而 ICMP 是**网络层**（Network Layer）协议，与 IP 协议处于同一层。ICMP 的主要目的是在主机和路由器之间传递网络层的控制信息和差错报文（如网络通断、主机不可达等），它是由操作系统的网络协议栈直接处理的，用于主机与主机之间的通信，而非进程与进程之间的通信，因此不需要使用端口号。

2.3.3 问题 3

问题： Examine one of the ping request packets... What are the ICMP type and code numbers? What other fields...? How many bytes...?

回答：观察捕获的 ICMP Echo Request（Ping 请求）数据包：

- ICMP 类型 (Type): 8
- 代码 (Code): 0
- 包含的其他字段：报文中还包含 Checksum（校验和）、Identifier（标识符）和 Sequence Number（序列号）。
- 各字段字节数：
 - Checksum: 2 字节 (2 bytes)
 - Identifier: 2 字节 (2 bytes)
 - Sequence Number: 2 字节 (2 bytes)

2.3.4 问题 4

问题： Examine the corresponding ping reply packet...

回答：观察对应的 ICMP Echo Reply（Ping 回复）数据包：

- ICMP 类型 (Type): 0
- 代码 (Code): 0
- 包含的其他字段：包含与请求报文相同的字段结构（Checksum、Identifier、Sequence Number）。
- 各字段字节数：



- Checksum: 2 字节 (2 bytes)
- Identifier: 2 字节 (2 bytes)
- Sequence Number: 2 字节 (2 bytes)

3 ICMP 的 traceroute 分析

3.1 traceroute 相关图片

以下是 traceroute 实验的相关截图：

```
C:\Users\liuhe>tracert www.bilibili.com

通过最多 30 个跃点跟踪
到 www.bilibili.com [120.192.82.76] 的路由:

 1    3 ms    4 ms    3 ms  10.150.160.1
 2    6 ms    3 ms    3 ms  10.70.3.1
 3    *      *      *    请求超时。
 4    4 ms    4 ms    4 ms  211.64.145.93
 5   11 ms    9 ms   10 ms  223.80.102.157
 6    *      *      *    请求超时。
 7    *      *      *    请求超时。
 8    *      *   39 ms  211.137.177.69
 9    *      *   18 ms  120.222.49.118
10   11 ms   17 ms   12 ms  120.222.151.54
11    *      *      *    请求超时。
12    *      *      *    请求超时。
13   24 ms   15 ms   15 ms  120.192.82.76

跟踪完成。
```

图 4: traceroute 命令执行结果

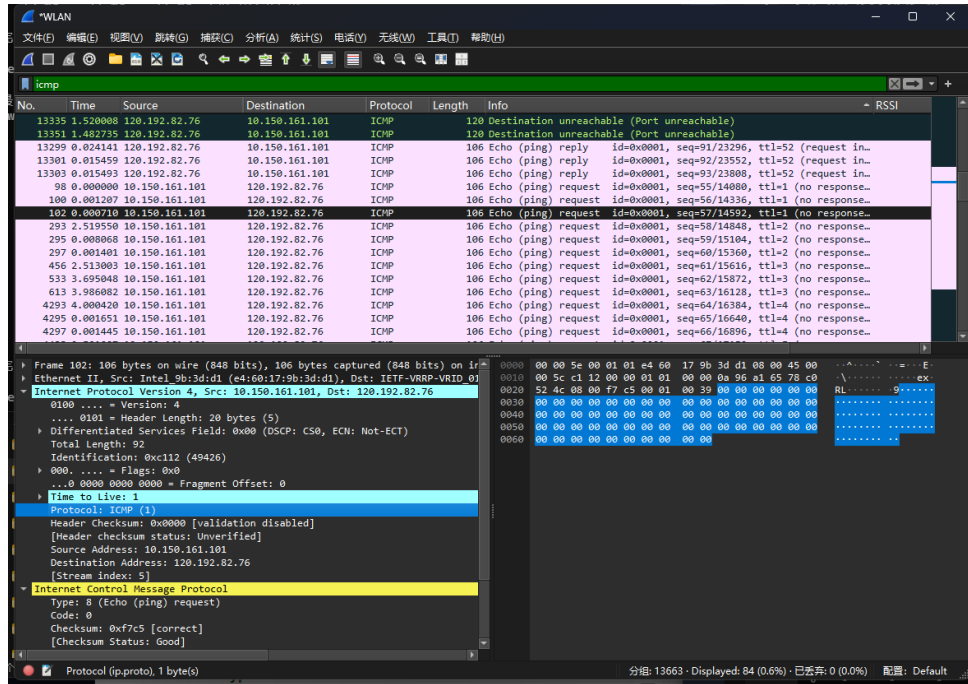


图 5: ICMP 错误数据包结构

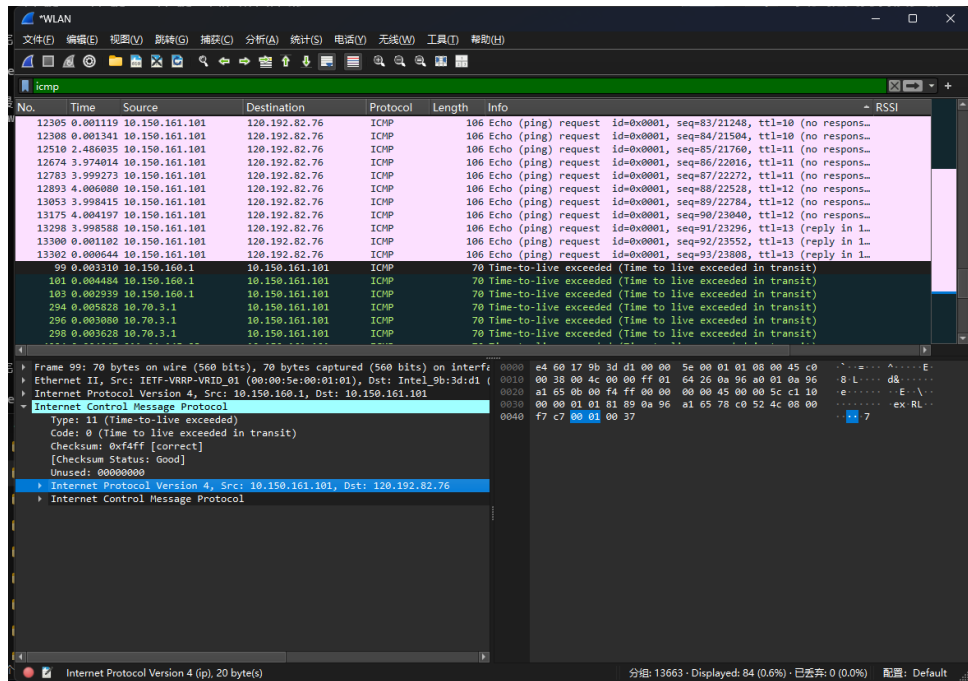


图 6: TTL 字段在 traceroute 中的设置（第二部分）

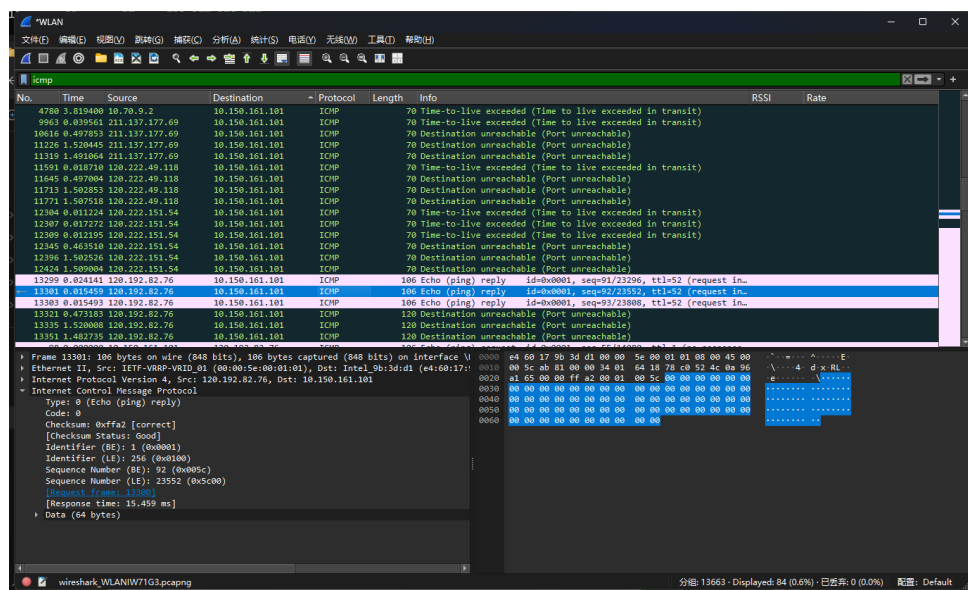


图 7: 最终 ICMP 响应包分析 (第二部分)

3.2 问题解答

3.2.1 问题 5

问题: What is the IP address of your host? What is the IP address of the target destination host?

回答: 根据截图显示:

- 本机 IP 地址 (Source) 是: 10.150.161.101
- 目标主机 IP 地址 (Destination) 是: 120.192.82.76 (即 www.bilibili.com 的 IP)

3.2.2 问题 6

问题: If ICMP sent UDP packets instead (as in Unix/Linux), would the IP protocol number still be 01 for the probe packets? If not, what would it be?

回答: 不会。如果发送的是 UDP 数据包, IP 头部中的 Protocol (协议) 字段将不再是 01 (ICMP)。它将变为 17 (十六进制为 0x11), 这是分配给 UDP 协议的编号。



3.2.3 问题 7

问题： Examine the ICMP echo packet in your screenshot. Is this different from the ICMP ping query packets in the first half of this lab? If yes, how so?

回答： 是的，有区别。虽然它们都是 **ICMP Echo Request** 类型 (Type 8, Code 0)，且都包含校验和、标识符和序列号，但关键的区别在于 IP 头部中的 **TTL (Time To Live)** 字段。

- 在实验前半部分的普通 Ping 操作中，TTL 通常是一个较大的默认值（如 64, 128 或 255）。
- 而在 Traceroute 的探测包（如我的截图 Frame 102）中，**TTL 被明确设置为 1**（随后的包会变为 2, 3 等）。这是 Traceroute 利用 TTL 超时机制来发现路径上路由器的原理。

3.2.4 问题 8

问题： Examine the ICMP error packet in your screenshot. It has more fields than the ICMP echo packet. What is included in those fields?

回答： 查看截图中的 Frame 99 (Type 11, Time-to-live exceeded)，该数据包包含的额外字段是“导致错误的原始数据包的”引用”。具体来说，它包含了：

1. 原始数据包的 **IP 头部** (IP Header)。
2. 原始数据包 **IP 载荷 (Payload)** 的前 8 个字节。

在我的 Wireshark 截图中，可以看到在 ICMP 协议层下方，嵌套显示了 *Internet Protocol Version 4* 和 *Internet Control Message Protocol*，这就是被包裹在错误消息里的原始探测包信息，用于让发送方识别是哪个探测包超时了。

3.2.5 问题 9

问题： Examine the last three ICMP packets received by the source host. How are these packets different from the ICMP error packets? Why are they different?

回答：

- 不同之处：最后收到的数据包（如截图 Frame 13301）是 **ICMP Echo Reply** (Type 0, Code 0)，而之前的中间数据包是 **ICMP Time-to-live Exceeded** (Type 11)。



- 原因：之前的包是因为 TTL 在中间路由器递减为 0，路由器丢弃包并报错。而最后的数据包**成功到达了最终目标主机**（120.192.82.76）。目标主机不会因为 TTL 到期而丢弃它，而是正常处理了 Echo Request 并返回了 Echo Reply 响应，标志着追踪完成。

3.2.6 问题 10

问题： Within the tracert measurements, is there a link whose delay is significantly longer than others? Refer to the screenshot in Figure 4, is there a link whose delay is significantly longer than others? On the basis of the router names, can you guess the location of the two routers on the end of this link?

回答：

- 关于延迟：在我的实验截图（tracert www.bilibili.com）中，并没有出现极端的延迟突增（例如从 10ms 跳变到 150ms 这种跨洋传输的特征），因为目标服务器在国内。不过，从第 5 跳（约 10ms）到第 8 跳（39ms）出现了相对明显的延迟增加，这可能涉及跨运营商或骨干网的传输。
- 关于位置推测：在我的 CMD 截图中，路由器仅显示了 IP 地址（如 211.137.177.69），并没有解析出域名（Hostname）。因此，**无法仅根据截图中的信息通过“路由器名称”来猜测它们的地理位置。**

4 实验结果与分析

4.1 ICMP 协议分析总结

通过前面的实验和分析，我们总结出 ICMP 协议在 ping 和 traceroute 工具中的关键作用：

- ICMP 作为网络层协议，主要用于传递网络层的控制信息和差错报文
- 在 ping 操作中，使用 ICMP Echo Request（Type 8）和 Echo Reply（Type 0）报文
- 在 traceroute 中，利用 IP 包的 TTL 字段和 ICMP Time Exceeded（Type 11）报文来发现路径



- ICMP 报文不包含端口号，因为它工作在传输层之上，用于主机间而非进程间通信

5 结论与展望

通过本次实验，我们深入了解了 ICMP 协议的工作原理，包括 ping 和 traceroute 工具的实现机制。实验结果表明，ICMP 协议在网络诊断和路径发现方面具有重要作用。